Password rules for Coup

How a player chooses a password, and what the strength bars mean

Version 2 · 27 October 2026 · Coup graduation project · Status: agreed proposal. The frontend checker follows these rules, and the register screen will use it; the backend will use the same rules when it is built.

1. Why we need password rules

A password protects a player's account, their Gold and Gems, and the skins they own. If someone guesses the password, they can take all of these.

The register screen will check the password while the player types. This gives quick help. But a browser check alone is not enough: a person can skip it with a tool such as Burp Suite and send a request straight to the server. So the server must check the same rules again. Both sides read one shared rules file, so the rules cannot drift apart.

2. The rules

Rule	What it means
At least 15 characters	Short passwords are easy to guess. Length gives the most protection.
At most 128 characters	This stops very long inputs that could slow down the server.
Four character types	At least one capital letter (A–Z), one small letter (a–z), one number (0–9), and one symbol, such as `! # $ @`.
Other characters are fine	Spaces and letters from any language (for example Arabic) are allowed. Each character counts once. They do not count as one of the four types.
Not a common password	Passwords from lists of common or leaked passwords are refused, also when only the capital letters are different or when letters are changed to look-alike symbols, such as `@` for a.
Nothing personal	The password must not contain the player's name or the name part of their email (before the @).
Strong enough	The strength bars must show at least **Fair**. A **Weak** password is refused.

When a password is refused, the player sees the reason, for example "Use at least 15 characters." or "Add a symbol, like ! or #."

3. Why we chose these rules

- **15 characters.** NIST says that a password which is the only way to log in must have at least 15 characters. It allows 8 characters only when the password is used together with another factor.[1] Players log in with a password only, so we use 15. A large study compared two rules: at least 16

characters with no other rule, and at least 8 characters with all four types and a dictionary check. The 16-character passwords had more entropy (44.67 against 34.30 bits). People also created them more easily and said they were easier to remember, but their measured recall was about the same.[6] So length is our main protection.

- **Four character types.** Many well-known sites ask for a mix of types: Microsoft asks for 3 of the 4 types,[7] and Apple[8] and the card payment standard PCI DSS[9] ask for letters and numbers. Players know this rule, so we ask for all four types.
- **Why we keep the other rules.** NIST and OWASP ASVS advise against type rules, because people answer them with patterns like "Password1!".[1][10] Our other rules catch many of these.
- **Which symbols count.** The 32 special characters on the OWASP list, such as `! # $ @`. The list also has the space, but we do not count it.[11]
- **Any character, counted once.** NIST says the system should accept all characters, including spaces and Unicode letters, normalize them (Unicode NFC), and count each one once.[1]
- **Blocklist.** NIST says the system must compare the whole password with a list of common, expected, or leaked passwords, and say why it was refused.[1] OWASP ASVS asks for at least the top 3,000 passwords.[10]
- **Personal words.** Attackers try a person's name and email first. The strength estimator also uses these words, so a password built from them gets a low score.[2][4]
- **No forced changes and no hints.** NIST says the system must not force regular password changes or store password hints. It asks for a new password only when the old one was stolen.[1]
- **Strength bars.** NIST says the system must help users choose a strong password.[1] The strength bars give this help while the player types.

4. The strength bars

Below the password field there are four bars and a word. They come from **zxcvbn**, a well-known strength estimator from Dropbox.[2][4] It estimates how many guesses an attacker would need, and it gives a score from 0 to 4.[4] We show the score like this:

What the player sees	Score	Guesses an attacker needs[4]	Result
–	–	–	Before typing. The hint says "At least 15 characters".
Weak	0–1	fewer than 1 million	**Refused.** One red bar.
Fair	2	fewer than 100 million	Accepted. Two yellow bars. This is the minimum.
Good	3	fewer than 10 billion	Accepted. Three green bars.
Strong	4	10 billion or more	Accepted. Four light purple bars, the game's "legendary" color.

> If any rule fails (for example too short, a missing type, common, or personal), the bars show **Weak**, even when the score is high. For example, `violet piano 47!` gets a high score, but it has no capital letter.

5. Examples

These examples come from the shared test file. Do not use them as real passwords.

Password	Bars	Why
`Short pass 1!`	Weak, refused	Only 13 characters.
`violet piano 47!`	Weak, refused	No capital letter.
`Violet piano 47 47`	Weak, refused	No symbol. A space is not a symbol.
`P@ssw0rdSt@ndard`	Weak, refused	A common password with capital letters and look-alike symbols.
`Qwerty123456789!`	Weak, refused	A keyboard pattern and a number sequence.
`Summer2026!2026!`	Fair	All four types, but built from a common word and a repeated year.
`Football20262026!`	Good	Long, but a common word and numbers.
`Violet piano 47!`	Strong	Words that do not belong together, a number, and a symbol.

> **Known limit.** Type rules make some people use patterns such as `Password12345678!`. The estimator rates this one Good, so it is accepted. When the server is built, it should also check a list of breached passwords, as OWASP ASVS asks at Level 2.[10]

6. Tips for players

- Avoid simple patterns, such as `Password1!` or `Qwerty123`.
- Use a different password for every website. A password manager can remember them for you.
- Never share your password. The Coup team will never ask for it.

7. How we keep the browser and the server the same

- **One rules file:** `integration/contracts/password-policy.json`. The frontend checker reads it now; the server will read it later.
- **One test file:** `integration/mocks/password-cases.json` with 28 test passwords. The browser tests and the future server tests must give the same result for each one.
- **Same estimator:** the browser uses zxcvbn 4.4.2.[4] The server should use zxcvbn-python 4.5.0, a port of the same version, with `max_length=128`, because its default limit is 72 characters.[5] We ran all

28 test passwords through both versions, and the results were the same.

- **Safe storage:** the server must hash passwords with Argon2id (at least 19 MiB of memory, 2 iterations, and 1 degree of parallelism) and a unique salt for each password.[3] It must not use bcrypt, because bcrypt reads only the first 72 bytes of a password.[3]
- **Limited attempts:** the server must limit failed logins. NIST allows at most 100 failed attempts in a row; a lower limit is better.[1]
- **No leaks:** the server must never log a password or send it back in a response.

References

1. National Institute of Standards and Technology (NIST). *Digital Identity Guidelines: Authentication and Authenticator Management*, NIST SP 800-63B-4, 2025. https://pages.nist.gov/800-63-4/sp800-63b.html (accessed 26 October 2026).
2. D. L. Wheeler. "zxcvbn: Low-Budget Password Strength Estimation." In *Proceedings of the 25th USENIX Security Symposium*, Austin, TX, USA, August 2016, pp. 157–173. https://www.usenix.org/conference/usenixsecurity16/technical-sessions/presentation/wheeler
3. OWASP Foundation. "Password Storage Cheat Sheet." OWASP Cheat Sheet Series. https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html (accessed 26 October 2026).
4. Dropbox. "zxcvbn: Low-Budget Password Strength Estimation," version 4.4.2. https://github.com/dropbox/zxcvbn
5. zxcvbn-python contributors. "zxcvbn-python," version 4.5.0. https://github.com/dwolfhub/zxcvbn-python
6. S. Komanduri, R. Shay, P. G. Kelley, M. L. Mazurek, L. Bauer, N. Christin, L. F. Cranor, and S. Egelman. "Of Passwords and People: Measuring the Effect of Password-Composition Policies." In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI '11)*, Vancouver, Canada, May 2011, pp. 2595–2604. https://doi.org/10.1145/1978942.1979321
7. Microsoft. "Self-service password reset policies – Microsoft Entra ID," section "Microsoft Entra password policies." Microsoft Learn. https://learn.microsoft.com/en-us/entra/identity/authentication/concept-sspr-policy (accessed 27 October 2026).
8. Apple. "Apple Account security overview." Apple Platform Security guide. https://support.apple.com/guide/security/apple-account-security-secbd5822138/web (accessed 27 October 2026).
9. PCI Security Standards Council. *Payment Card Industry Data Security Standard*, version 4.0.1, June 2024, Requirement 8.3.6. https://www.pcisecuritystandards.org/document_library/
10. OWASP Foundation. *Application Security Verification Standard*, version 5.0, section V6.2 "Password Security" (requirements 6.2.1, 6.2.4, 6.2.5, and 6.2.12). https://github.com/OWASP/ASVS/blob/master/5.0/en/0x15-V6-Authentication.md (accessed 27 October 2026).
11. OWASP Foundation. "Password Special Characters." OWASP Community. https://community.owasp.org/password-special-characters (accessed 27 October 2026).